

CCF CSP CERTIFICATION

代 码 速 查 手 册

HHY C++ Light Theme • 语法高亮

覆盖范围 STL / 搜索 / 图论 / DP / 数学 / 数据结构

页数 约 30 页 • A4 双面打印

版本 2025 年 7 月 9 日

主题配色说明

■ 关键字 ■ 类型 ■ 字符串 ■ 注释 ■ 数字

目录

1 基础框架模板	4
1.1 万能头 + 类型别名 + 常量	4
1.2 快读	4
1.3 main 框架 (CSP 必写)	4
2 STL 速查	5
2.1 vector	5
2.2 set / map	5
2.3 multiset (允许重复的有序集合)	5
2.4 next_permutation (全排列神器)	6
2.5 priority_queue (堆)	6
2.6 pair / tuple / bitset	6
2.7 deque / queue / stack	6
2.8 哈希表 (数组模拟)	7
3 字符串处理	7
3.1 string 常用操作	7
3.2 字符串哈希 (O(1) 判子串相等)	8
3.3 KMP 字符串匹配	8
3.4 Trie 树 (字符串统计)	9
4 基础算法	9
4.1 sort + 去重 + 二分查找	9
4.2 nth_element (O(n) 求第 k 小)	10
4.3 手写排序 (快排 + 归并)	10
4.4 二分答案 (两大模板)	11
4.5 浮点数二分	11
4.6 贪心经典模型	12
4.7 区间合并	12
5 数学	12
5.1 GCD / LCM / 快速幂 / 素数筛	12
5.2 质因数分解 + 质数判断 + 约数	13
5.3 高斯消元	14
5.4 欧拉函数	14
5.5 扩展欧几里得 + Lucas + 卡特兰数	15
6 搜索	16
6.1 DFS 连通块 (Flood Fill)	16
6.2 DFS 回溯 + BFS 最短路	16

7 图论	17
7.1 邻接表存图 + Dijkstra	17
7.2 SPFA + Bellman-Ford + Floyd	17
7.3 并查集 + 拓扑排序 + Kruskal	18
7.4 Tarjan 强连通分量	19
7.5 Prim + 二分图 + 匈牙利	20
8 树	21
8.1 树的存储、遍历、DFS 序	21
8.2 LCA (倍增法)	21
8.3 树形 DP	22
9 动态规划	22
9.1 背包全家桶	22
9.2 LIS / LCS / 最大子段和	23
9.3 区间 DP / 状压 DP / 分组背包	23
10 数据结构	24
10.1 线段树 (区间修改 + 查询)	24
10.2 树状数组	25
10.3 单调栈 / 单调队列	25
10.4 分块	26
11 常用技巧	26
11.1 前缀和 (一维/二维)	26
11.2 差分 (一维/二维)	26
11.3 离散化 + 双指针 + 位运算	27
11.4 高精度运算	27
11.5 矩阵运算	28
12 复杂度速查与考试策略	29
12.1 数据规模与时间复杂度对照	29
12.2 题型策略	29
12.3 历年真题考点速查	29
13 模板改写指南 (看到题 → 改模板)	29
13.1 基础算法改写	30
13.2 数据结构改写	30
13.3 图论改写	30
13.4 数学改写	30
13.5 快速对应表 (题面 → 算法)	31

1 基础框架模板

1.1 万能头 + 类型别名 + 常量

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3f;
const int N = 200005;
```

1.2 快读

```
inline int read() {
    int x = 0, f = 1; char c = getchar();
    while (c < '0' || c > '9') {
        if (c == '-') f = -1;
        c = getchar();
    }
    while (c >= '0' && c <= '9') {
        x = x * 10 + c - '0';
        c = getchar();
    }
    return x * f;
}
```

1.3 main 框架 (CSP 必写)

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
#endif

    int T = 1; // cin >> T;
    while (T--) {
        solve();
    }
    return 0;
}
```

考前 30 秒 Checklist

1. 数组开够 N 了吗? ($n=10^5$ 时开 $2 \times 10^5+5$)
2. 并查集初始化了吗? (`for i fa[i]=i`)
3. map 查询用 `count()` 了吗? (别用 `mp[x]` 查存在性)
4. `getline` 前有 `cin>>` 时, `getchar()` 吃掉换行了吗?
5. 输出格式对了吗? (空格、换行、大小写)
6. `ios::sync_with_stdio(false); cin.tie(nullptr);` 写了吗?
7. pq 的 cmp 方向跟 sort 是反的, 确认对了吗?
8. `stoi` 转大数会 RE, 该用 `stoll` 或手写转换吗?

2 STL 速查

2.1 vector

```
vector<int> v;
v.push_back(x);
v.pop_back();
v.front(); v.back();
v.clear();

vector<int> v(n, 0);           // n个0
vector<vector<int>> g(n);      // 邻接表

// 排序+去重
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
```

2.2 set / map

```
set<int> s;                     // 有序  $O(\log n)$ 
unordered_set<int> us;         // 无序  $O(1)$ 
s.insert(x); s.erase(x);
s.find(x) != s.end();
s.count(x);
s.lower_bound(x);              // 第一个  $\geq x$ 

map<string, int> mp;            // 有序键值对
unordered_map<string, int> ump; // 无序更快
mp[key] = val;                 // 插入/修改 (会新建键!)
mp.count(key);                 // 查存在性用这个, 别用 mp[x]
for (auto& [k, v] : mp) { }    // C++17 遍历
```

2.3 multiset (允许重复的有序集合)

```
multiset<int> ms;
ms.insert(x);                  // 插入 (允许重复)
ms.erase(ms.find(x));          // 只删一个 x
ms.erase(x);                   // 删所有 x!
auto it = ms.lower_bound(x);    // 第一个  $\geq x$ 
```

2.4 next_permutation (全排列神器)

```
sort(a.begin(), a.end());
do {
    // 处理当前排列
} while (next_permutation(a.begin(), a.end()));
// prev_permutation 为上一个排列
```

2.5 priority_queue (堆)

```
// 大根堆(默认)
priority_queue<int> pq;

// 小根堆
template<typename T>
using min_pq =
    priority_queue<T, vector<T>, greater<T>>;
min_pq<int> pq;

// 自定义结构体小顶堆
struct Node { int d, u; };
struct Cmp {
    bool operator()(const Node& a,
                    const Node& b) const {
        return a.d > b.d;
    }
};
// 注意: pq的Cmp与sort相反!
// sort: Cmp(a,b)=true 表示a排前面
// pq: Cmp(a,b)=true 表示a排后面
priority_queue<Node, vector<Node>, Cmp> pq;
```

2.6 pair / tuple / bitset

```
pair<int, int> p = {1, 2};
cout << p.first << " " << p.second;
vector<pair<int, int>> v;
sort(v.begin(), v.end());

tuple<int, int, int> t = {1, 2, 3};
auto [a, b, c] = t; // C++17 解构

// bitset
bitset<1005> b;
b.set(i); b.reset(i); b.flip(i);
b.test(i); b.count(); b.any(); b.none();
bitset<1005> b1 | b2; // 位运算交并
```

2.7 deque / queue / stack

```
// deque 双端队列
deque<int> dq;
dq.push_back(x); dq.push_front(x);
dq.pop_back(); dq.pop_front();
dq.front(); dq.back();
dq[i]; // 随机访问 O(1)
```

```
// queue 队列 (BFS标配)
queue<int> q;
q.push(x); q.pop(); q.front();

// stack 栈 (DFS/递归辅助)
stack<int> st;
st.push(x); st.pop(); st.top();
```

2.8 哈希表（数组模拟）

```
// 拉链法
int h[N], e[N], ne[N], idx;
void insert(int x) {
    int k = (x % N + N) % N;
    e[idx] = x;
    ne[idx] = h[k];
    h[k] = idx++;
}
bool find(int x) {
    int k = (x % N + N) % N;
    for (int i = h[k]; i != -1; i = ne[i])
        if (e[i] == x) return true;
    return false;
}

// 开放寻址法
int h[N]; // null = 0x3f3f3f3f
int find(int x) {
    int t = (x % N + N) % N;
    while (h[t] != null && h[t] != x) {
        t++;
        if (t == N) t = 0;
    }
    return t;
}
```

3 字符串处理

3.1 string 常用操作

```
string s, t;
cin >> s;
getline(cin, s); // 整行输入
// 前面用过 cin >> 时, 先 getchar() 吃掉残留换行符!
// cin >> n; getchar(); getline(cin, s);

s.size(); // 长度
s.substr(pos, len); // 子串
s.find(t); // 查找, npos=未找到
s.find(t, pos); // 从pos开始查找
s.rfind(t); // 从右查找
s.insert(pos, t); // 插入
s.erase(pos, len); // 删除
s.replace(pos, len, t); // 替换
s.find_first_of("abc"); // 第一个属于"abc"的位置
```

```

s.find_first_not_of("0123456789"); // 第一个非数字

// 大小写转换
transform(s.begin(), s.end(),
          s.begin(), ::tolower);

// 类型转换
int x = stoi(s);
ll y = stoll(s);
// stoi 溢出会 RE, 大数用 stoll 或手写转换
double d = stod(s);
s = to_string(x);

// stringstream 分词
#include <sstream>
stringstream ss(s);
string token;
while (ss >> token) { }

```

3.2 字符串哈希 (O(1) 判子串相等)

```

// 将字符串看作P进制数, 自然溢出 (ULL)
typedef unsigned long long ULL;
const int P = 131; // 或13331
ULL hl[N], hr[N], p[N];

void init(const string& s) {
    int n = s.size();
    p[0] = 1;
    for (int i = 1; i <= n; i++) {
        hl[i] = hl[i-1] * P + s[i-1];
        hr[i] = hr[i-1] * P + s[n-i];
        p[i] = p[i-1] * P;
    }
}

// 获取子串 [l, r] 的哈希值 (1-indexed)
ULL get(int l, int r) {
    return hl[r] - hl[l-1] * p[r-l+1];
}

// 判断 [l1, r1] 和 [l2, r2] 是否相等
bool same(int l1, int r1,
          int l2, int r2) {
    return get(l1, r1) == get(l2, r2);
}

```

3.3 KMP 字符串匹配

```

// s: 文本串 (1-indexed), p: 模式串 (1-indexed)
int ne[N]; // next数组

// 求next数组
for (int i = 2, j = 0; i <= m; i++) {
    while (j && p[i] != p[j+1])
        j = ne[j];
    if (p[i] == p[j+1]) j++;
    ne[i] = j;
}

```



```
// 匹配
for (int i = 1, j = 0; i <= n; i++) {
    while (j && s[i] != p[j + 1])
        j = ne[j];
    if (s[i] == p[j + 1]) j++;
    if (j == m) {
        printf("%d ", i - m + 1);
        j = ne[j]; // 继续匹配
    }
}

// 求循环节: 若  $n \% (n - ne[n]) == 0$ 
// 循环节长度 =  $n - ne[n]$ 
```

3.4 Trie 树 (字符串统计)

```
// son: 子节点, cnt: 经过次数
int son[N][26], cnt[N], idx;

void insert(const char* str) {
    int p = 0;
    for (int i = 0; str[i]; i++) {
        int u = str[i] - 'a';
        if (!son[p][u])
            son[p][u] = ++idx;
        p = son[p][u];
    }
    cnt[p]++;
}

int query(const char* str) {
    int p = 0;
    for (int i = 0; str[i]; i++) {
        int u = str[i] - 'a';
        if (!son[p][u]) return 0;
        p = son[p][u];
    }
    return cnt[p]; // 出现次数
}
```

4 基础算法

4.1 sort + 去重 + 二分查找

```
sort(a.begin(), a.end()); // 升序
sort(a.begin(), a.end(),
    greater<int>()); // 降序
sort(a + 1, a + 1 + n); // 数组

// 自定义排序
sort(v.begin(), v.end(),
    [](const T& a, const T& b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y > b.y;
    });
```

```
// 去重 (先排序)
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()),
        v.end());

// 二分查找
lower_bound(v.begin(), v.end(), x); // 第一个>=x
upper_bound(v.begin(), v.end(), x); // 第一个>x
binary_search(v.begin(), v.end(), x); // 是否存在
```

4.2 nth_element (O(n) 求第 k 小)

```
// 求第 k 小 (0-indexed)
nth_element(a.begin(),
            a.begin() + k,
            a.end());

// 此时 a[k] 就是第 k 小的元素

// 第 k 大
nth_element(a.begin(),
            a.begin() + k,
            a.end(),
            greater<int>());
```

4.3 手写排序 (快排 + 归并)

```
// 快速排序
void quickSort(int a[], int l, int r) {
    if (l >= r) return;
    int x = a[(l + r) >> 1];
    int i = l - 1, j = r + 1;
    while (i < j) {
        do i++; while (a[i] < x);
        do j--; while (a[j] > x);
        if (i < j) swap(a[i], a[j]);
    }
    quickSort(a, l, j);
    quickSort(a, j + 1, r);
}

// 归并排序 (顺便求逆序对)
ll mergeSort(int a[], int l, int r,
             int tmp[]) {
    if (l >= r) return 0;
    int mid = (l + r) >> 1;
    ll inv = mergeSort(a, l, mid, tmp);
    inv += mergeSort(a, mid + 1, r, tmp);
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j])
            tmp[k++] = a[i++];
        else {
            tmp[k++] = a[j++];
            inv += mid - i + 1;
        }
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    for (int i = l; i <= r; i++) a[i] = tmp[i - l];
    return inv;
}
```

```

while (i <= mid) tmp[k++] = a[i++];
while (j <= r)    tmp[k++] = a[j++];
for (i = l, j = 0; i <= r; i++, j++)
    a[i] = tmp[j];
return inv;
}

```

4.4 二分答案（两大模板）

```

// 模板A: 求最小值（最小化最大值）
int l = 0, r = 1e9, ans = r;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) {
        ans = mid;
        r = mid - 1;
    } else {
        l = mid + 1;
    }
}

// 模板B: 求最大值（最大化最小值）
int l = 0, r = 1e9, ans = l;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) {
        ans = mid;
        l = mid + 1;
    } else {
        r = mid - 1;
    }
}

```

4.5 浮点数二分

```

// 浮点数二分：精度控制
const double EPS = 1e-7;
double l = 0, r = 1e9, ans = r;
while (r - l > EPS) {
    double mid = (l + r) / 2;
    if (check(mid)) {
        ans = mid; r = mid;
    } else {
        l = mid;
    }
}

// 固定迭代次数（推荐）
double l = 0, r = 1e9;
for (int i = 0; i < 100; i++) {
    double mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid;
}

```

4.6 贪心经典模型

```
// 活动安排 (按结束时间排序)
sort(a.begin(), a.end(),
     [](Act x, Act y) { return x.r < y.r; });
int cnt = 0, last = -1;
for (auto& [l, r] : a)
    if (l >= last) { cnt++; last = r; }

// 哈夫曼 / 合并果子 (小根堆)
min_pq<ll> pq; // 所有值入堆
ll ans = 0;
while (pq.size() > 1) {
    ll a = pq.top(); pq.pop();
    ll b = pq.top(); pq.pop();
    ans += a + b;
    pq.push(a + b);
}
```

4.7 区间合并

```
// 将所有存在交集的区间合并
vector<pair<int,int>> merge(
    vector<pair<int,int>>& segs) {
    vector<pair<int,int>> res;
    sort(segs.begin(), segs.end());
    int st = -2e9, ed = -2e9;
    for (auto& [l, r] : segs) {
        if (ed < l) {
            if (st != -2e9)
                res.push_back({st, ed});
            st = l; ed = r;
        } else {
            ed = max(ed, r);
        }
    }
    if (st != -2e9) res.push_back({st, ed});
    return res;
}
```

5 数学

5.1 GCD / LCM / 快速幂 / 素数筛

```
ll gcd(ll a, ll b) {
    return b ? gcd(b, a % b) : a;
}
ll lcm(ll a, ll b) {
    return a / gcd(a, b) * b;
}
// 直接用内置函数: __gcd(a, b)
// C++内置, 省4行不会错

// 快速幂
ll qpow(ll a, ll b, ll mod) {
    ll res = 1 % mod;
```

```

    a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

// 欧拉筛 / 线性筛
const int MAXN = 1e6 + 5;
int primes[MAXN], cnt = 0;
bool isComp[MAXN];
void sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (!isComp[i]) primes[cnt++] = i;
        for (int j = 0;
             j < cnt && i * primes[j] <= n;
             j++) {
            isComp[i * primes[j]] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

```

5.2 质因数分解 + 质数判断 + 约数

```

vector<pair<ll, int>> factorize(ll n) {
    vector<pair<ll, int>> fac;
    for (ll i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            int c = 0;
            while (n % i == 0) {
                n /= i; c++;
            }
            fac.push_back({i, c});
        }
    }
    if (n > 1) fac.push_back({n, 1});
    return fac;
}

// 单数质数判断
bool isPrime(ll n) {
    if (n < 2) return false;
    for (ll i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}

// 求所有约数
vector<ll> getDivisors(ll n) {
    vector<ll> res;
    for (ll i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            res.push_back(i);
            if (i != n / i)
                res.push_back(n / i);
        }
    }
}

```

```

    }
    return res;
}

// 约数个数
ll divisorCount(
    const vector<pair<ll,int>>& fac) {
    ll cnt = 1;
    for (auto& [p, e] : fac) cnt *= (e + 1);
    return cnt;
}

```

5.3 高斯消元

```

const double EPS = 1e-9;
int gauss(vector<vector<double>>& a) {
    int n = a.size(), m = a[0].size();
    int r = 0;
    for (int c = 0; c < m && r < n; c++) {
        int t = r;
        for (int i = r; i < n; i++)
            if (fabs(a[i][c]) > fabs(a[t][c]))
                t = i;
        if (fabs(a[t][c]) < EPS) continue;
        swap(a[t], a[r]);
        for (int i = r + 1; i < n; i++)
            if (fabs(a[i][c]) > EPS)
                for (int j = m - 1; j >= c; j--)
                    a[i][j] -= a[r][j]
                        * a[i][c] / a[r][c];

        r++;
    }
    return r; // 矩阵的秩
}

```

5.4 欧拉函数

```

// 单个数的欧拉函数
int phi(int x) {
    int res = x;
    for (int i = 2; i <= x / i; i++)
        if (x % i == 0) {
            res = res / i * (i - 1);
            while (x % i == 0) x /= i;
        }
    if (x > 1) res = res / x * (x - 1);
    return res;
}

// 筛法求 1~n 所有欧拉函数
int euler[N]; bool st[N];
int primes[N], cnt;
void getEuler(int n) {
    euler[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!st[i]) {
            primes[cnt++] = i;

```

```

        euler[i] = i - 1;
    }
    for (int j = 0;
        primes[j] <= n / i; j++) {
        st[primes[j] * i] = true;
        if (i % primes[j] == 0) {
            euler[primes[j]*i]
                = euler[i] * primes[j];
            break;
        }
        euler[primes[j]*i]
            = euler[i] * (primes[j] - 1);
    }
}
}

```

5.5 扩展欧几里得 + Lucas + 卡特兰数

```

// exgcd: 求  $ax + by = \gcd(a, b)$  的解
int exgcd(int a, int b, int& x, int& y) {
    if (!b) { x = 1; y = 0; return a; }
    int d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}

// 求a在模mod下的乘法逆元
int inv(int a, int mod) {
    int x, y;
    exgcd(a, mod, x, y);
    return (x % mod + mod) % mod;
}

// Lucas定理
int C(int a, int b, int p) {
    if (a < b) return 0;
    ll x = 1, y = 1;
    for (int i = a, j = 1; j <= b;
        i--, j++) {
        x = x * i % p;
        y = y * j % p;
    }
    return x * qpow(y, p - 2, p) % p;
}

int lucas(ll a, ll b, int p) {
    if (a < p && b < p) return C(a, b, p);
    return (ll)C(a % p, b % p, p)
        * lucas(a / p, b / p, p) % p;
}

// 卡特兰数
//  $Cat(n) = C(2n, n) / (n+1)$ 
// 合法括号序列 / 出栈序列 / 二叉树形态
ll catalan(int n) {
    return Comb(2*n, n)
        * inv(n+1, mod) % mod;
}

```

6 搜索

6.1 DFS 连通块 (Flood Fill)

```
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

void dfs(int x, int y) {
    vis[x][y] = true;
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];
        if (nx < 0 || nx >= n) continue;
        if (ny < 0 || ny >= m) continue;
        if (vis[nx][ny]) continue;
        if (g[nx][ny] == '#') continue;
        dfs(nx, ny);
    }
}

// 统计连通块：外层循环，
// 遇到未访问就 dfs
```

6.2 DFS 回溯 + BFS 最短路

```
// 排列生成
void dfs(int step) {
    if (step == n) {
        // 记录答案
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!used[i]) {
            used[i] = true;
            path[step] = i;
            dfs(step + 1);
            used[i] = false; // 回溯
        }
    }
}

// 组合生成 (start去重)
void dfs(int step, int start) {
    if (step == k) { /* 输出 */ return; }
    for (int i = start; i < n; i++) {
        path[step] = a[i];
        dfs(step + 1, i + 1);
    }
}

// BFS 最短路 (无权图)
vector<int> dist(n, -1);
void bfs(int s) {
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
```



```

        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
}
}

```

7 图论

7.1 邻接表存图 + Dijkstra

```

// 带权图
vector<vector<pair<int,int>>> g(n);
void add(int u, int v, int w) {
    g[u].push_back({v, w});
    // g[v].push_back({u, w}); // 无向图
}

// Dijkstra  $O((V+E)\log V)$ , 非负权
vector<ll> dijkstra(int s) {
    vector<ll> dist(n, LINF);
    dist[s] = 0;
    min_pq<pair<ll, int>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        for (auto& [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

7.2 SPFA + Bellman-Ford + Floyd

```

// SPFA (负权边)
bool spfa(int s) {
    queue<int> q;
    fill(dist, dist + n, INF);
    dist[s] = 0;
    q.push(s); inq[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        inq[u] = false;
        for (auto& [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    q.push(v); inq[v] = true;
                    if (++cnt[v] >= n)

```

```

        return false; // 负环
    }
}

return true;
}

// Bellman-Ford (限制边数k)
bool bellmanFord(int s, int limit) {
    fill(dist, dist + n, INF);
    dist[s] = 0;
    for (int i = 0; i < limit; i++) {
        memcpy(backup, dist, sizeof dist);
        for (int j = 0; j < m; j++) {
            auto& e = edges[j];
            dist[e.b] = min(dist[e.b],
                            backup[e.a] + e.w);
        }
    }
    return true;
}

// Floyd (多源最短路)  $O(n^3)$ 
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][k] + dist[k][j]
                < dist[i][j])
                dist[i][j]
                    = dist[i][k] + dist[k][j];

```

7.3 并查集 + 拓扑排序 + Kruskal

```

// 并查集
int fa[N];
int find(int x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}
void unite(int x, int y) {
    fa[find(x)] = find(y);
}
bool same(int x, int y) {
    return find(x) == find(y);
}

// 初始化: for(int i=0; i<n; i++) fa[i]=i;

// 维护集合大小的并查集
int fa[N], sz[N];
void initUF(int n) {
    for (int i = 0; i <= n; i++) {
        fa[i] = i; sz[i] = 1;
    }
}
bool unite(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return false;
    if (sz[x] < sz[y]) swap(x, y);
    fa[y] = x; sz[x] += sz[y];
}

```

```

    return true;
}

// 带权并查集 (维护到祖先距离)
int fa[N], d[N];
int find(int x) {
    if (fa[x] == x) return x;
    int root = find(fa[x]);
    d[x] += d[fa[x]];
    return fa[x] = root;
}
void unite(int x, int y, int w) {
    int fx = find(x), fy = find(y);
    if (fx == fy) return;
    fa[fy] = fx;
    d[fy] = d[x] - d[y] + w;
}

```

```

// 拓扑排序 (Kahn算法)
bool topoSort(int n, vector<int>& res) {
    queue<int> q;
    for (int i = 1; i <= n; i++)
        if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    return res.size() == n;
}

// Kruskal 最小生成树
struct Edge { int u, v, w; };
vector<Edge> edges;
sort(edges.begin(), edges.end(),
    [](Edge a, Edge b) { return a.w < b.w; });
ll kruskal() {
    ll ans = 0;
    int cnt = 0;
    for (auto& e : edges) {
        if (find(e.u) != find(e.v)) {
            unite(e.u, e.v);
            ans += e.w;
            if (++cnt == n - 1) break;
        }
    }
    return ans;
}

```

7.4 Tarjan 强连通分量

```

int dfn[N], low[N], stk[N], ins[N];
int scc[N], sccCnt, top, times;

void tarjan(int u) {

```

```

    dfn[u] = low[u] = ++times;
    stk[++top] = u;
    ins[u] = 1;
    for (int v : g[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (ins[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) {
        sccCnt++;
        do {
            int x = stk[top--];
            ins[x] = 0;
            scc[x] = sccCnt;
        } while (stk[top + 1] != u);
    }
}
}

```

7.5 Prim + 二分图 + 匈牙利

```

// Prim MST (稠密图  $O(n^2)$ )
int g[N][N], dist[N]; bool st[N];
int prim() {
    memset(dist, 0x3f, sizeof dist);
    int res = 0;
    for (int i = 0; i < n; i++) {
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;
        if (i && dist[t] == INF) return INF;
        if (i) res += dist[t];
        st[t] = true;
        for (int j = 1; j <= n; j++)
            dist[j] = min(dist[j], g[t][j]);
    }
    return res;
}

// 二分图判定 (染色法)
int color[N]; // -1未染色, 0/1
bool dfs(int u, int c) {
    color[u] = c;
    for (int v : g[u]) {
        if (color[v] == -1) {
            if (!dfs(v, !c)) return false;
        } else if (color[v] == c)
            return false;
    }
    return true;
}

// 匈牙利算法 (二分图最大匹配)
int match[N]; bool vis[N];
bool find(int u) {

```

```

    for (int v : g[u]) {
        if (vis[v]) continue;
        vis[v] = true;
        if (match[v] == 0 || find(match[v])) {
            match[v] = u;
            return true;
        }
    }
    return false;
}

```

8 树

8.1 树的存储、遍历、DFS 序

```

vector<vector<int>> tree(n);
tree[u].push_back(v);
tree[v].push_back(u);

// DFS 遍历树
void dfsTree(int u, int fa) {
    for (int v : tree[u]) {
        if (v != fa) {
            dfsTree(v, u);
        }
    }
}

// DFS 序
int tin[N], tout[N], euler[N], timer = 0;
void dfs(int u, int fa) {
    tin[u] = ++timer;
    euler[timer] = u;
    for (int v : tree[u])
        if (v != fa) dfs(v, u);
    tout[u] = timer;
}

// 子树 u = euler[tin[u]..tout[u]]

```

8.2 LCA (倍增法)

```

const int LOG = 20;
int up[N][LOG], dep[N];

void dfsLCA(int u, int fa) {
    up[u][0] = fa;
    for (int i = 1; i < LOG; i++)
        up[u][i] = up[up[u][i-1]][i-1];
    for (int v : tree[u]) {
        if (v != fa) {
            dep[v] = dep[u] + 1;
            dfsLCA(v, u);
        }
    }
}

```

```
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int i = LOG - 1; i >= 0; i--)
        if (dep[up[u][i]] >= dep[v])
            u = up[u][i];
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--)
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    return up[u][0];
}
```

8.3 树形 DP

```
// dp[u] 表示以u为根的子树的最优值
void treeDP(int u, int fa) {
    dp[u] = 初始值;
    for (int v : tree[u]) {
        if (v != fa) {
            treeDP(v, u);
            dp[u] = merge(dp[u], dp[v]);
        }
    }
}
```

9 动态规划

9.1 背包全家桶

```
// 01背包 (容量倒序)
int dp[V + 1] = {0};
for (int i = 0; i < n; i++)
    for (int j = V; j >= w[i]; j--)
        dp[j] = max(dp[j],
                     dp[j - w[i]] + v[i]);

// 完全背包 (容量正序)
for (int i = 0; i < n; i++)
    for (int j = w[i]; j <= V; j++)
        dp[j] = max(dp[j],
                     dp[j - w[i]] + v[i]);

// 多重背包 (二进制拆分)
vector<int> nw, nv;
for (int i = 0; i < n; i++) {
    int cnt = s[i];
    for (int k = 1; k <= cnt; k *= 2) {
        cnt -= k;
        nw.push_back(w[i] * k);
        nv.push_back(v[i] * k);
    }
    if (cnt) {
        nw.push_back(w[i] * cnt);
        nv.push_back(v[i] * cnt);
    }
}
```

```

    }
}
// 对  $nw, nv$  跑 01 背包

```

9.2 LIS / LCS / 最大子段和

```

// LIS  $O(n \log n)$ 
template<typename T>
int lis(const vector<T>& a) {
    vector<T> tail;
    for (T x : a) {
        auto it = lower_bound(
            tail.begin(), tail.end(), x);
        if (it == tail.end())
            tail.push_back(x);
        else
            *it = x;
    }
    return tail.size();
}
// 非严格递增: lower_bound 改 upper_bound

// LCS
int lcs(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(
        n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1])
                ? dp[i-1][j-1] + 1
                : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}

// 最大子段和  $O(n)$ 
ll maxSubarray(const vector<int>& a) {
    ll sum = 0, ans = LLONG_MIN;
    for (int x : a) {
        sum = max((ll)x, sum + x);
        ans = max(ans, sum);
    }
    return ans;
}

```

9.3 区间 DP / 状压 DP / 分组背包

```

// 区间DP
for (int len = 2; len <= n; len++)
    for (int i = 1; i + len - 1 <= n; i++) {
        int j = i + len - 1;
        for (int k = i; k < j; k++)
            dp[i][j] = max(dp[i][j],
                dp[i][k] + dp[k+1][j] + cost);
    }

// 状压DP ( $n \leq 20$ )

```

```

for (int s = 0; s < (1 << n); s++)
    for (int i = 0; i < n; i++)
        if (s >> i & 1)
            for (int j = 0; j < n; j++)
                if (!(s >> j & 1))
                    dp[s|1<<j][j] = max(
                        dp[s|1<<j][j],
                        dp[s][i] + w[i][j]);

// 分组背包
for (int k = 0; k < group; k++)
    for (int j = V; j >= 0; j--)
        for (auto& [w, v] : group[k])
            if (j >= w)
                dp[j] = max(dp[j],
                    dp[j - w] + v);

```

10 数据结构

10.1 线段树（区间修改 + 查询）

```

struct SegTree {
    struct Node { ll sum, lz; } tr[N * 4];

    void pushup(int p) {
        tr[p].sum = tr[p*2].sum
            + tr[p*2+1].sum;
    }
    void pushdown(int p, int l, int r) {
        if (!tr[p].lz) return;
        int mid = (l + r) / 2;
        tr[p*2].sum += tr[p].lz
            * (mid - l + 1);
        tr[p*2+1].sum += tr[p].lz
            * (r - mid);
        tr[p*2].lz += tr[p].lz;
        tr[p*2+1].lz += tr[p].lz;
        tr[p].lz = 0;
    }
    void build(int p, int l, int r) {
        tr[p].lz = 0;
        if (l == r) {
            tr[p].sum = a[l]; return;
        }
        int mid = (l + r) / 2;
        build(p*2, l, mid);
        build(p*2+1, mid+1, r);
        pushup(p);
    }
    void update(int p, int l, int r,
        int L, int R, ll v) {
        if (L <= l && r <= R) {
            tr[p].sum += v * (r - l + 1);
            tr[p].lz += v;
            return;
        }
        pushdown(p, l, r);
    }

```



```

        int mid = (l + r) / 2;
        if (L <= mid)
            update(p*2, l, mid, L, R, v);
        if (R > mid)
            update(p*2+1, mid+1, r, L, R, v);
        pushup(p);
    }
    ll query(int p, int l, int r,
            int L, int R) {
        if (L <= l && r <= R)
            return tr[p].sum;
        pushdown(p, l, r);
        int mid = (l + r) / 2;
        ll res = 0;
        if (L <= mid)
            res += query(p*2, l, mid, L, R);
        if (R > mid)
            res += query(p*2+1, mid+1, r, L, R);
        return res;
    }
};

```

10.2 树状数组

```

int c[N];
int lowbit(int x) { return x & (-x); }
void add(int x, int v) {
    for (; x <= n; x += lowbit(x)) c[x] += v;
}
int query(int x) {
    int s = 0;
    for (; x; x -= lowbit(x)) s += c[x];
    return s;
}
// 区间 [l, r] 和 = query(r) - query(l - 1)

```

10.3 单调栈 / 单调队列

```

// 单调栈：找左边第一个比它小的数
int tt = 0;
for (int i = 1; i <= n; i++) {
    while (tt && stk[tt] >= a[i]) tt--;
    // stk[tt] 就是左边第一个比 a[i] 小的
    stk[++tt] = a[i];
}

// 单调队列：滑动窗口最小值
int hh = 0, tt = -1;
for (int i = 0; i < n; i++) {
    if (hh <= tt && q[hh] < i - k + 1)
        hh++;
    while (hh <= tt && a[q[tt]] >= a[i])
        tt--;
    q[++tt] = i;
    if (i >= k - 1)
        printf("%d ", a[q[hh]]);
}

```

10.4 分块

```
int belong[N], L[N], R[N], tag[N];
void build() {
    int block = sqrt(n);
    int num = n / block;
    if (n % block) num++;
    for (int i = 1; i <= num; i++) {
        L[i] = (i - 1) * block + 1;
        R[i] = min(i * block, n);
        for (int j = L[i]; j <= R[i]; j++)
            belong[j] = i;
    }
}
// 区间加：完整块打tag，不完整块暴力
// 区间查：完整块用tag+块内和，不完整块暴力
```

11 常用技巧

11.1 前缀和（一维/二维）

```
// 一维
vector<ll> pre(n + 1, 0);
for (int i = 0; i < n; i++)
    pre[i + 1] = pre[i] + a[i];
// 区间[l,r]和 = pre[r+1] - pre[l]

// 二维
vector<vector<ll>> s(n+1, vector<ll>(m+1,0));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        s[i][j] = a[i][j] + s[i-1][j]
            + s[i][j-1] - s[i-1][j-1];
// 子矩阵(x1,y1)到(x2,y2)
ll sub = s[x2][y2] - s[x1-1][y2]
    - s[x2][y1-1] + s[x1-1][y1-1];
```

11.2 差分（一维/二维）

```
// 一维差分：区间[l, r] += val
vector<ll> diff(n + 2, 0);
void rangeAdd(int l, int r, ll val) {
    diff[l] += val; diff[r + 1] -= val;
}
// 还原
ll cur = 0;
for (int i = 0; i < n; i++) {
    cur += diff[i]; a[i] = cur;
}

// 二维差分
// 子矩阵(x1,y1)到(x2,y2) += d
diff[x1][y1] += d;
diff[x2+1][y1] -= d;
diff[x1][y2+1] -= d;
diff[x2+1][y2+1] += d;
```

11.3 离散化 + 双指针 + 位运算

```
// 标准离散化 (排序+去重+二分)
vector<int> alls = a;           // 1.拷贝
sort(alls.begin(), alls.end());
alls.erase(unique(alls.begin(),
                  alls.end()),
            alls.end());

int getId(int x) {              // 2.获取下标
    return lower_bound(alls.begin(),
                      alls.end(), x)
        - alls.begin();
}                               // +1 如果从1开始

// 双指针 / 滑动窗口
int l = 0, ans = 0;
for (int r = 0; r < n; r++) {
    window.add(a[r]);
    while (!check(window)) {
        window.remove(a[l]); l++;
    }
    ans = max(ans, r - l + 1);
}

// 位运算常用技巧
x & 1;                          // 判断奇偶
x & (x - 1);                    // 去掉最低位的1
x & (-x);                       // 获取最低位的1
x | (1 << i);                   // 将第i位置1
x & ~(1 << i);                 // 将第i位置0
x ^ (1 << i);                   // 翻转第i位
(x >> i) & 1;                   // 获取第i位
__builtin_popcount(x);         // 1的个数
for (int s = m; s; s = (s-1) & m) // 枚举子集
```

11.4 高精度运算

```
// 高精度加 (逆序存储)
vector<int> add(vector<int>& A,
               vector<int>& B) {
    vector<int> C;
    int t = 0;
    for (int i = 0;
         i < A.size() || i < B.size() || t;
         i++) {
        if (i < A.size()) t += A[i];
        if (i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    return C;
}

// 高精度减 (A >= B)
vector<int> sub(vector<int>& A,
               vector<int>& B) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size(); i++) {
```

```

        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        t = (t < 0) ? 1 : 0;
    }
    while (C.size() > 1 && C.back() == 0)
        C.pop_back();
    return C;
}

// 高精度除以低精度
vector<int> div(vector<int>& A,
               int b, int& r) {
    vector<int> C; r = 0;
    for (int i = A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0)
        C.pop_back();
    return C;
}

```

11.5 矩阵运算

```

// 矩阵乘法  $C = A * B$ 
vector<vector<ll>> mul(
    const vector<vector<ll>>& A,
    const vector<vector<ll>>& B,
    int mod) {
    int n = A.size(), m = B[0].size();
    int p = B.size();
    vector<vector<ll>> C(n,
        vector<ll>(m, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            for (int k = 0; k < p; k++)
                C[i][j] = (C[i][j]
                    + A[i][k] * B[k][j]) % mod;
    return C;
}

// 矩阵转置
vector<vector<ll>> transpose(
    const vector<vector<ll>>& A) {
    int n = A.size(), m = A[0].size();
    vector<vector<ll>> B(m,
        vector<ll>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            B[j][i] = A[i][j];
    return B;
}

```

12 复杂度速查与考试策略

12.1 数据规模与时间复杂度对照

数据规模	可接受复杂度	算法
$n \leq 20$	$O(2^n)$	状压 DP、枚举子集
$n \leq 100$	$O(n^3)$	Floyd、 $O(n^2)$ DP
$n \leq 1000$	$O(n^2)$	$O(n^2)$ DP、邻接矩阵
$n \leq 10^5$	$O(n \log n)$	sort、线段树、堆
$n \leq 10^6$	$O(n)$	线性扫描、双指针
$n \leq 10^9$	$O(\log n)$	二分、数学公式

12.2 题型策略

题型	目标	建议时间
T1 水题	必拿 100 分	10–20min
T2 基础	必拿 100 分	20–30min
T3 大模拟	争取 70–100 分	60–90min
T4 图论/DP	争取 50–80 分	30–60min
T5 压轴	暴力拿 10–30 分	剩余时间

提示

时间分配关键：T1+T2 合计控制在 **40 分钟** 内完成！T3 大模拟非常耗时，做好写 **2 小时** 左右的心理准备（含调试），T4+T5 剩余 **约 1 小时**。T3 是拉开分数差距的关键题，务必留足时间。

12.3 历年真题考点速查

时间	T1	T2	T3	T4	T5
2024.06	循环模拟	矩阵操作	大模拟 +PQ	贪心 + 背包	线段树/分块
2024.03	map 计数	集合交并	高斯消元	set+ 离散化 +PQ	链表 +DFS 序
2023.12	循环模拟	质因数分解	DFS 树上搜索	分块 + 前缀和	状压 DP
2023.09	循环	前缀积 + 前缀和	后缀表达式	set	树形 DP+ 线段树
2023.05	string+map	矩阵乘法	字符串 + 位运算	暴力枚举	最短路 + 状压 DP
2023.03	循环	二分答案	递归 +bitset	线段树 + 离散化	差分 + 分治
2022.12	循环	拓扑排序	循环 + 矩阵	启发式合并	线段树 + 高精度

注意

常见失分点：数组越界（开够 N）、输出格式多空格/少换行、忘记 ios 加速导致 TLE、二分死循环（mid 计算方式）、DFS 忘记回溯、并查集没初始化、stoi 溢出。

13 模板改写指南（看到题 → 改模板）

提示

核心思路：多数题目不需要从零写算法，而是在基础模板上**增加额外信息**或**调整判定逻辑**。

13.1 基础算法改写

看到这类题	基础模板	改写方向
求第 k 大/小的数	快速排序	快速选择：只递归 pivot 半边
求逆序对个数	归并排序	合并时累加 mid-i+1
最大值最小/最小值最大	二分查找	二分答案 +check(mid)
最长无重复子串	双指针	滑动窗口
值域大但数据量少	离散化	离散化 + 树状数组/线段树
区间覆盖总长度	区间合并	合并后累加长度

13.2 数据结构改写

看到这类题	基础模板	改写方向
带距离/关系的连通块合并	并查集	带权并查集
食物链、对立关系判断	并查集	种类并查集（开 3 倍空间）
滑动窗口最值	单调队列	模板直接套用
找左边第一个比它小的	单调栈	维护递增栈
字符串循环节	KMP	$n\%(n-\text{ne}[n])=0$
最大异或对	Trie 树	二进制 Trie，走相反位

13.3 图论改写

看到这类题	基础模板	改写方向
求次短路	Dijkstra	$\text{dist1}+\text{dist2}$
最短路带边数限制	Bellman-Ford	限制迭代次数
判断负环	SPFA	$\text{cnt}[x]>=n$
传递闭包	Floyd	$\text{bool}+d[i][j]=d[i][k]\&\&d[k][j]$
次小生成树	Kruskal	枚举非树边替换
二分图最大匹配	匈牙利	match 数组即方案

13.4 数学改写

看到这类题	基础模板	改写方向
递推式加速	快速幂	矩阵快速幂
解 $ax+by=c$	exgcd	先求 gcd 判断有解
解异或方程组	高斯消元	加减改异或
组合数 n,m 很大	组合数	Lucas 定理
合法括号序列计数	卡特兰数	$\text{Cat}(n)=C(2n,n)/(n+1)$

13.5 快速对应表（题面 → 算法）

看到这类题	用什么	真题对照
连通/合并	并查集	28-T4 高速公路
带距离的连通块	带权并查集	31-T5 阻击
依赖排序	拓扑排序	28-T2 训练计划
区间修改 + 查询	线段树/分块	34-T5 哥德尔机、32-T4 宝藏
化学方程式	高斯消元	33-T3 化学方程式
树上子树合并	DFS 序 + 并查集	33-T5 文件夹合并
矩阵/坐标变换	矩阵前缀积	30-T2 矩阵运算、31-T2 坐标变换
字符串编码	map+pq	34-T3 文本分词、30-T3 解压缩
最短路 + 状态压缩	最短路 + 状压 DP	30-T5 闪耀巡航、32-T5 彩色路径
区间分配/回收	set 维护区间	33-T4 十滴水
最大值最小	二分答案	29-T2 垦田计划